

Trabalho Parcial - Ciência de Dados e Inteligência Artificial

Gabriel Ceni Programação Avançada - 2026.2

1. Instalação e Importação de Bibliotecas

```
# Instalação pacotes necessários em modo silencioso (-q)
!pip install ucimlrepo plotly scikit-learn -q
```

```
#
import pandas as pd
import pandas.io.formats.style
import numpy as np
import hashlib

from typing import Optional, Literal

# Dados
from ucimlrepo import fetch_ucirepo
```

Import Plots

```
# Visualização
import plotly.express as px
import plotly.io as pio
import plotly.graph_objects as go

from plotly.subplots import make_subplots
```

Import MLs - SKLearn

```
# Machine Learning
from sklearn.model_selection import train_test_split
from sklearn.model_selection import StratifiedKFold

from sklearn.compose import ColumnTransformer
from sklearn.pipeline import Pipeline

from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import RobustScaler, OneHotEncoder

from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.decomposition import PCA

import joblib
```

```
print(pd.__version__)
```

2.2.3

Definições Globais

```
# Ajuste do Plotly para Colab
pio.renderers.default = 'colab'

# Definição de Cores/Temas
WINE_COLORS = {
    'red': '#722F37',    # Wine Plum
    'white': '#EDE8D0'   # Vanilla Crea
}

SPOILED_COLORS = {
    'red': '#5C3B39',    # Chocolate Plum
    'white': '#895129'   # Saddle Brown
}
```

```
def style_wine_df(
    df: pd.DataFrame,
    n_rows: Optional[int] = None,
    method: Literal["head", "sample"] = "head",
    quality_threshold: int = 6,
    random_state: Optional[int] = 33
) -> pandas.io.formats.style.Styler:
    """
    Estiliza um DataFrame de vinhos colorindo as linhas de acordo com a cor e a qualidade.

    - Usa WINE_COLORS quando quality >= quality_threshold
    - Usa SPOILED_COLORS quando quality < quality_threshold

    Parameters
    -----
    df : pd.DataFrame
        DataFrame contendo as colunas 'color' e 'quality'.
    n_rows : int, optional
        Quantidade de linhas a exibir. Se None, exibe todas.
    method : {"head", "sample"}, default="head"
        Método de seleção das linhas:
        - "head": pega as primeiras n_rows
        - "sample": pega n_rows aleatórias
    quality_threshold : int, default=6
        Limite de qualidade para decidir entre cores normais e "spoiled".
    random_state : int, optional
        Semente para o sample (garante reprodutibilidade). Só usado quando method="sample".

    Returns
    -----
    pd.io.formats.style.Styler
        Objeto Styler pronto para ser exibido com display().
    """
    if n_rows is None:
        data = df
    elif method == "sample":
        data = df.sample(n=n_rows, random_state=random_state)
    else: # method == "head"
        data = df.head(n_rows)

    def apply_color(row: pd.Series) -> list[str]:
        is_spoiled = row['quality'] < quality_threshold
        color_map = SPOILED_COLORS if is_spoiled else WINE_COLORS

        bg_color = color_map.get(row['color'], '#FFFFFF')
        text_color = 'white' if row['color'] == 'red' else 'black'

        return [f'background-color: {bg_color}; color: {text_color}'] * len(row)

    return (
        data.style
        .apply(apply_color, axis=1)
        .set_properties(**{'text-align': 'center'})
        .format(precision=2)
        .set_caption("Wine Dataset - Colorido por tipo e qualidade")
    )
```

✓ 2. Visualização Prévia e Tratamento de Dados

```
# Carregar o dataset Wine Quality
wine_quality = fetch_ucirepo(id=186)
```

```
wine_df = wine_quality.data.original
# display(wine_df.head(3))
display(style_wine_df(wine_df, n_rows=3, method="head"))
print("=== Amostra ===")
# display(wine_df.head(20))
display(style_wine_df(wine_df, n_rows=20, method="sample"))
print("=== Describe ===")
display(wine_df.describe())
```

```
Wine Dataset — Colorido por tipo e qualidade
fixed_acidity volatile_acidity citric_acid residual_sugar chlorides free_sulfur_dioxide total_sulfur_dioxide density
0 7.40 0.70 0.00 1.90 0.08 11.00 34.00 1.11
1 7.80 0.88 0.00 2.60 0.10 25.00 67.00 1.11
2 7.80 0.76 0.04 2.30 0.09 15.00 54.00 1.11
=== Amostra ===

Wine Dataset — Colorido por tipo e qualidade
fixed_acidity volatile_acidity citric_acid residual_sugar chlorides free_sulfur_dioxide total_sulfur_dioxide density
5530 6.20 0.30 0.28 1.60 0.04 28.00 106.00
3603 7.40 0.26 0.31 2.40 0.04 58.00 178.00
4126 7.30 0.16 0.35 1.50 0.04 29.00 108.00
2582 5.70 0.36 0.34 4.20 0.03 21.00 77.00
5451 5.90 0.20 0.23 1.50 0.04 38.00 93.00
5223 6.60 0.36 0.28 6.10 0.03 12.00 93.00
3938 7.00 0.23 0.26 7.20 0.04 21.00 90.00
1363 8.00 0.83 0.27 2.00 0.08 11.00 63.00
5728 6.30 0.34 0.31 6.00 0.02 18.00 68.00
757 8.10 0.87 0.00 2.20 0.08 10.00 31.00
3227 8.20 0.20 0.49 3.50 0.06 14.00 108.00
1552 6.30 0.68 0.01 3.70 0.10 32.00 54.00
4919 7.50 0.29 0.36 15.70 0.05 29.00 124.00
2943 7.30 0.23 0.37 1.90 0.04 51.00 165.00
3077 7.90 0.22 0.24 4.60 0.04 39.00 159.00
23 8.50 0.49 0.11 2.30 0.08 9.00 67.00
1598 6.00 0.31 0.47 3.60 0.07 18.00 42.00
2868 8.40 0.35 0.56 13.80 0.05 55.00 190.00
4587 7.20 0.19 0.31 6.30 0.03 17.00 103.00
1859 6.90 0.12 0.35 5.40 0.05 36.00 108.00
=== Describe ===
fixed_acidity volatile_acidity citric_acid residual_sugar chlorides free_sulfur_dioxide total_sulfur_dioxide density
count 6497.000000 6497.000000 6497.000000 6497.000000 6497.000000 6497.000000 6497.000000
mean 7.215307 0.339666 0.318633 5.443235 0.056034 30.525319 115.744574
std 1.296434 0.164636 0.145318 4.757804 0.035034 17.749400 56.521855
min 3.800000 0.080000 0.000000 0.600000 0.009000 1.000000 6.000000
25% 6.400000 0.230000 0.250000 1.800000 0.038000 17.000000 77.000000
50% 7.000000 0.290000 0.310000 3.000000 0.047000 29.000000 118.000000
75% 7.700000 0.400000 0.390000 8.100000 0.065000 41.000000 156.000000
max 15.900000 1.580000 1.660000 65.800000 0.611000 289.000000 440.000000
```

2.1 Tratamento de Dados

Verificação de valores nulos e duplicados.

```
# Verificação de valores nulos
print("Linhas Nulas?\n", wine_df.isnull().sum())
print("Linhas Nulas?\n", wine_df.isnull().sum().sum(), " / ", wine_df.shape[0])

Linhas Nulas?
fixed_acidity      0
volatile_acidity   0
citric_acid        0
residual_sugar     0
chlorides          0
free_sulfur_dioxide 0
total_sulfur_dioxide 0
density            0
pH                 0
sulphates          0
```

```
alcohol      0
quality      0
color        0
dtype: int64
Linhas Nulas?
0 / 6497
```

```
# Verificação e remoção de dados duplicados
```

```
print("Valores Duplicados?\n", wine_df.duplicated().sum(), " / ", wine_df.shape[0])
wine_uniq_df = wine_df.drop_duplicates()
print("DataFrame Sem Duplicados?\n", wine_uniq_df.duplicated().sum(), " / ", wine_uniq_df.shape[0])
```

```
Valores Duplicados?
1177 / 6497
DataFrame Sem Duplicados?
0 / 5320
```

A presença de bastante linhas duplicadas leva a decidir entre:

- Remover linhas duplicadas:
 - **Prevenção de Vazamento de Dados** (*Data Leakage*) - Evitar cópias exatas do mesmo vinho.
 - **Redução de Viés** - Impedir que o algoritmo dê um peso desproporcional a um perfil de vinho específico.
 - **Segurança da Avaliação** - Garantir que o modelo seja testado de forma honesta contra dados que ele realmente nunca viu.
- Manter linhas duplicadas:
 - **Possibilidade de Coincidência Legítima** - Ausência de identificadores não permitem obter certeza se são os mesmos vinhos ou apenas similares.
 - **Preservação da Distribuição Natural** - Se as duplicatas representam a realidade, então equivalem à frequência real que a IA deva aprender.

✓ 2.2 Atributos (features)

- **fixed_acidity**: Ácidos que não evaporam facilmente. São fundamentais para garantir a acidez base e a estrutura geral do vinho.
- **volatile_acidity**: Quantidade de ácido acético. Em níveis elevados, confere ao vinho um cheiro e sabor indesejáveis de vinagre.
- **citric_acid**: Presente em pequenas quantidades, o ácido cítrico adiciona frescor e notas de sabor ao vinho.
- **residual_sugar**: O açúcar que permanece no vinho após o fim da fermentação. Define o nível de doçura da bebida.
- **chlorides**: A quantidade de sal (cloretos) dissolvida no vinho.
- **free_sulfur_dioxide**: Dióxido de enxofre em estado livre. Atua como agente antimicrobiano e evita a oxidação precoce do vinho.
- **total_sulfur_dioxide**: A soma do dióxido de enxofre livre com o ligado a outras moléculas. Em concentrações muito altas, afeta negativamente o cheiro e o sabor.
- **density**: A densidade do líquido. Varia diretamente de acordo com a quantidade de açúcar e inversamente em relação ao teor de álcool.
- **pH**: Mede quão ácido ou básico o vinho é. A escala vai de 0 a 14, mas a grande maioria dos vinhos situa-se entre 3.0 e 4.0.
- **sulphates**: Aditivos que contribuem para a manutenção dos níveis de dióxido de enxofre, funcionando como conservantes antioxidantes.
- **alcohol**: O percentual de álcool por volume presente na bebida.

```
# features
X_df = wine_uniq_df.drop(columns=['quality', 'color'])
print("=== Head ===")
display(X_df.sample(10))
print("=== Describe ===")
display(X_df.describe())
print("=== Info ===")
print(X_df.info())
```

```

=== Head ===
      fixed_acidity  volatile_acidity  citric_acid  residual_sugar  chlorides  free_sulfur_dioxide  total_sulfur_dioxide  c
1747             6.3              0.340        0.33             4.60        0.034              19.0              80.0
268              6.9              0.540        0.04             3.00        0.077              7.0              27.0
537              8.1              0.825        0.24             2.10        0.084              5.0              13.0
6305             6.1              0.220        0.40             1.85        0.031              25.0             111.0
6082             5.8              0.280        0.18             1.20        0.058              7.0             108.0
662              7.2              0.570        0.06             1.60        0.076              9.0              27.0
4579             6.1              0.380        0.42             5.00        0.016              31.0             113.0
2303             5.6              0.250        0.26             3.60        0.037              18.0             115.0
6173             6.2              0.200        0.26             9.70        0.030              39.0             102.0
1631             7.4              0.250        0.36             2.05        0.050              31.0             100.0

=== Describe ===
      fixed_acidity  volatile_acidity  citric_acid  residual_sugar  chlorides  free_sulfur_dioxide  total_sulfur_dioxide
count    5320.000000      5320.000000  5320.000000   5320.000000  5320.000000      5320.000000      5320.000000C
mean       7.215179        0.344130   0.318494     5.048477   0.056690      30.036654      114.109023
std       1.319671        0.168248   0.147157     4.500180   0.036863      17.805045       56.774223
min       3.800000        0.080000   0.000000     0.600000   0.009000       1.000000       6.000000C
25%       6.400000        0.230000   0.240000     1.800000   0.038000      16.000000      74.000000C
50%       7.000000        0.300000   0.310000     2.700000   0.047000      28.000000     116.000000C
75%       7.700000        0.410000   0.400000     7.500000   0.066000      41.000000     153.250000C
max      15.900000        1.580000   1.660000    65.800000   0.611000     289.000000     440.000000C

=== Info ===
<class 'pandas.core.frame.DataFrame'>
Index: 5320 entries, 0 to 6496
Data columns (total 11 columns):
 #   Column              Non-Null Count  Dtype
---  -
 0   fixed_acidity       5320 non-null  float64
 1   volatile_acidity    5320 non-null  float64
 2   citric_acid         5320 non-null  float64
 3   residual_sugar      5320 non-null  float64
 4   chlorides           5320 non-null  float64
 5   free_sulfur_dioxide 5320 non-null  float64
 6   total_sulfur_dioxide 5320 non-null  float64
 7   density             5320 non-null  float64
 8   pH                  5320 non-null  float64
 9   sulphates           5320 non-null  float64
10  alcohol             5320 non-null  float64
dtypes: float64(11)
memory usage: 498.8 KB
None

```

2.3 Variáveis Dependentes (targets)

- **Quality:** Uma nota de 0 a 10 para a qualidade do vinho.
- **Color:** Cor/Tipo do vinho.

```
display(wine_df[['quality', 'color']].head(7))
```

	quality	color
0	5	red
1	5	red
2	5	red
3	6	red
4	5	red
5	5	red
6	5	red

```

# targets -> quality
y_quality_df = wine_uniq_df['quality']
print("=== Head ===")

```

```
display(y_quality_df.sample(10))
print("=== Describe ===")
display(y_quality_df.describe())
print("=== Info ===")
print(y_quality_df.info())
```

```
=== Head ===
```

	quality
419	5
2954	5
3024	6
120	5
4865	6
2472	3
4080	6
4358	6
5811	4
3194	6

```
dtype: int64
```

```
=== Describe ===
```

	quality
count	5320.000000
mean	5.795677
std	0.879772
min	3.000000
25%	5.000000
50%	6.000000
75%	6.000000
max	9.000000

```
dtype: float64
```

```
=== Info ===
```

```
<class 'pandas.core.series.Series'>
Index: 5320 entries, 0 to 6496
Series name: quality
Non-Null Count  Dtype
-----
5320 non-null   int64
dtypes: int64(1)
memory usage: 83.1 KB
None
```

```
# targets -> color
y_color_df = wine_uniq_df['color']
print("=== Head ===")
display(y_color_df.sample(10))
print("=== Describe ===")
display(y_color_df.describe())
print("=== Info ===")
print(y_color_df.info())
```

```

=== Head ===
   color
6314  white
6181  white
2155  white
1541   red
5178  white
3772  white
449    red
5497  white
4119  white
937   red

dtype: object
=== Describe ===
   color
count    5320
unique      2
top      white
freq     3961

dtype: object
=== Info ===
<class 'pandas.core.series.Series'>
Index: 5320 entries, 0 to 6496
Series name: color
Non-Null Count  Dtype
-----
5320 non-null   object
dtypes: object(1)
memory usage: 83.1+ KB
None

```

3. Análise Exploratória de Dados (EDA)

3.1 Análise Univariada

Avaliação de cada variável isoladamente (*features* e *targets*).

```

# Features
# Grid
fig_features = make_subplots(rows=4, cols=3, subplot_titles=X_df.columns)

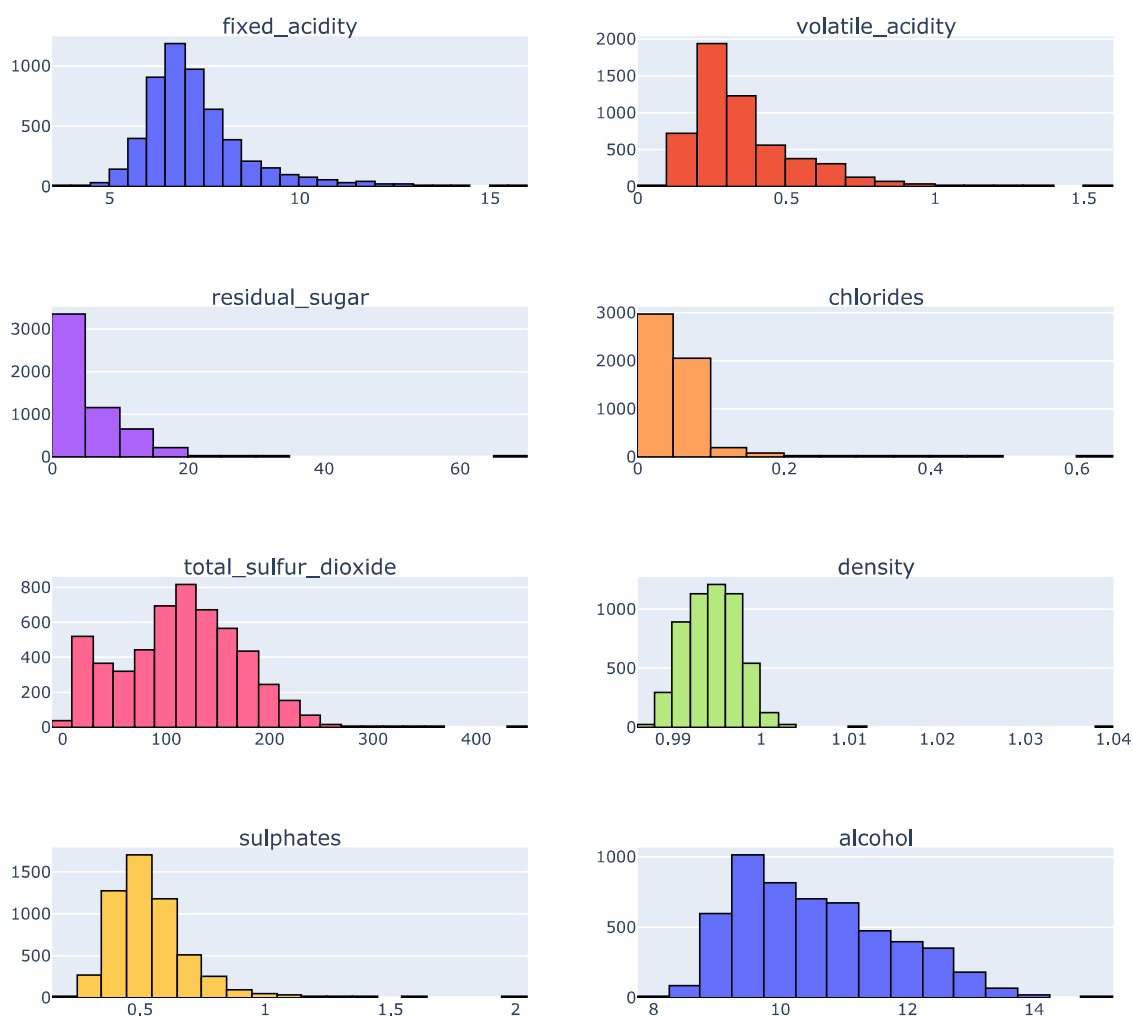
row = 1
col = 1
for feature in X_df.columns:
    fig_features.add_trace(
        go.Histogram(
            x=X_df[feature],
            name=feature,
            nbinsx=30,
            marker=dict(line=dict(color='black', width=1))
        ),
        row=row,
        col=col
    )
    col += 1
    if col > 3:
        col = 1
        row += 1

fig_features.update_layout(
    title_text="Distribuição das Variáveis Físico-Químicas (Features)",
    showlegend=False,
    # Colab: largura flexível e fixar a altura
    autosize=True,
    height=900
)

```

```
# config='responsive' permite que o Plotly preencha a largura da célula no notebook
fig_features.show(config={'responsive': True})
```

Distribuição das Variáveis Físico-Químicas (Features)



- **pH** e **density** apresentam dados similares a uma distribuição normal.
- **residual_sugar**, **chlorides** e **free_sulfur_dioxide** apresentam assimetria (*skewness*) positiva, com presença de medições extremas (*outliers*) à direita.
- **fixed_acidity**, **volatile_acidity**, **citric_acid** e **sulphates** apresentam assimetria positiva (cauda longa à direita) e presença de *outliers* na cauda superior.
- **alcohol** apresenta assimetria positiva, com pico de concentração em torno de 9,5% - 10% e queda gradual até 14%.

O fato de múltiplas variáveis apresentarem assimetria positiva (*right-skewness*) simultaneamente sugere fortes hipóteses de relacionamento entre as *features* e indica padrões estruturais no processo de geração dos dados

```
# Targets
## Quality
fig_quality = px.histogram(
    y_quality_df,
    x='quality',
    title='Distribuição da Qualidade (Target de Regressão/Classificação)',
    text_auto=True,
    color_discrete_sequence=['#832232'] # Tom de vinho
)

fig_quality.update_traces(marker_line_color='black', marker_line_width=1)

fig_quality.update_layout(
    autosize=True,
    height=400
)
```

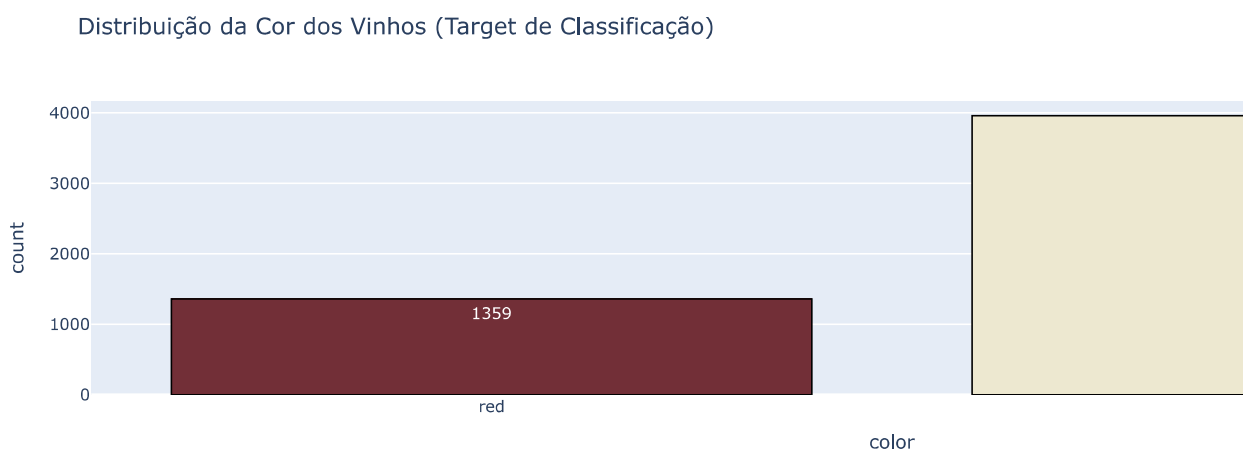
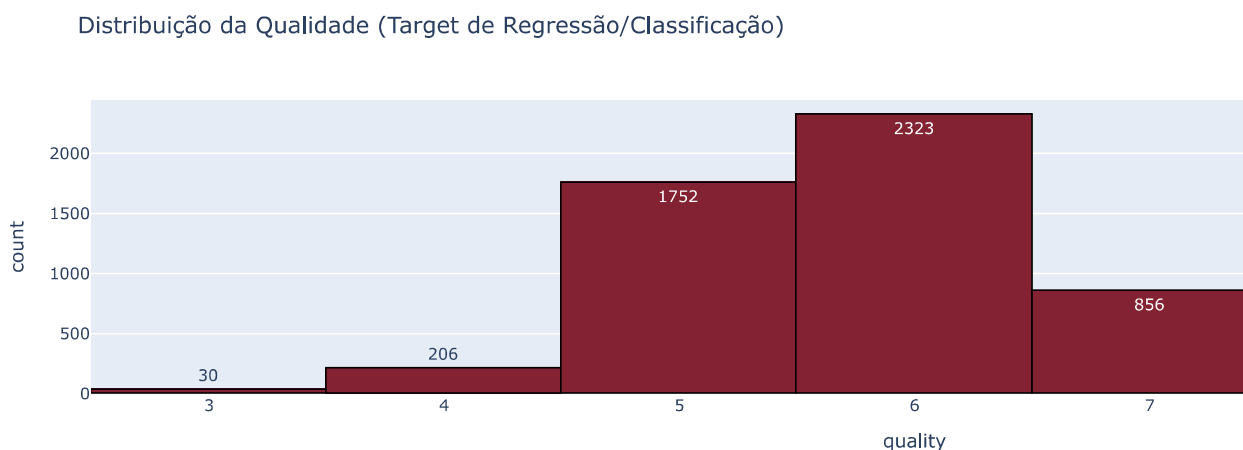
```
fig_quality.show(config={'responsive': True})

## Color
fig_color = px.histogram(
    y_color_df,
    x='color',
    title='Distribuição da Cor dos Vinhos (Target de Classificação)',
    text_auto=True,
    color='color',
    color_discrete_map=WINE_COLORS
)

fig_color.update_traces(marker_line_color='black', marker_line_width=1)

fig_color.update_layout(
    autosize=True,
    height=400
)

fig_color.show(config={'responsive': True})
```



- `quality` apresenta uma distribuição discreta com aparente simetria em torno do valor central 6 (concentrando a maior parte das amostras entre as notas 5 e 7), mas sofre de um nítido **desbalanceamento de classes** nas extremidades.

✓ 3.2 Análise de Valores Discrepantes (Outliers)

Evidenciar pontos fora da curva.

✓ 3.2.1 ph e density

```
grupo_1 = ['pH', 'density']

for feature in grupo_1:
    fig = px.histogram(
        wine_uniq_df,
```

```
x=feature,
marginal='violin',
title=f'Grupo 1 - Variáveis Contínuas de Tendência Central: {feature}',
color_discrete_sequence=['#2E8B57'],
opacity=0.8
)

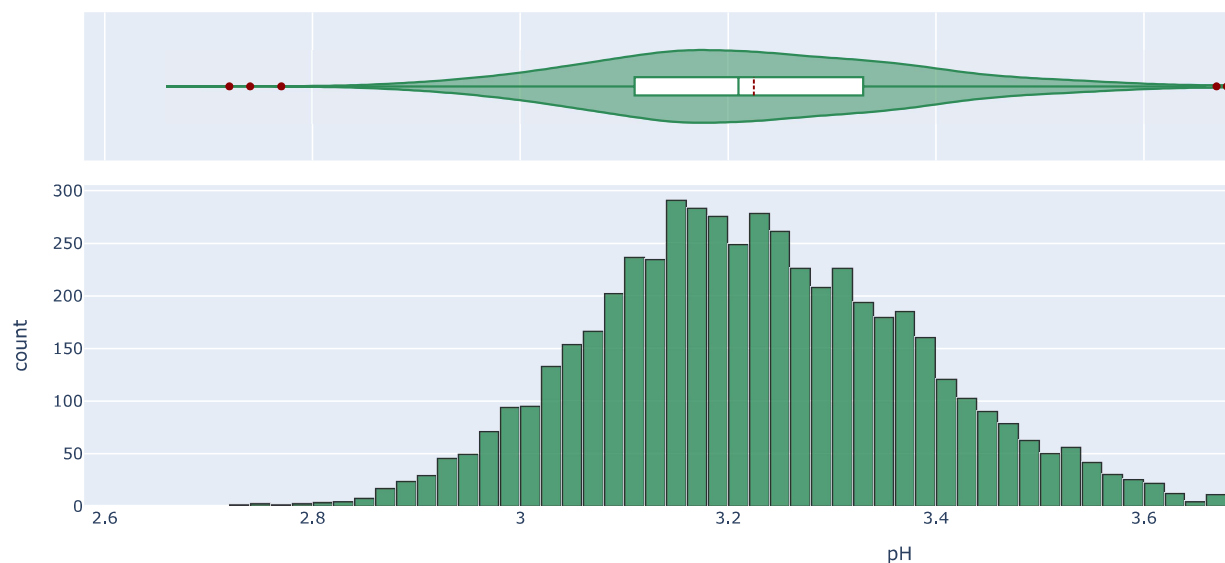
# Estilização Violin
fig.update_traces(
    box_visible=True,
    box_fillcolor='white',
    meanline_visible=True,
    meanline_color='darkred',
    line_color='#2E8B57',
    line_width=1.5,
    marker_symbol='circle',
    marker_color='#8B0000',
    selector=dict(type='violin')
)

# Estilização Histogram
fig.update_traces(
    marker_line_color='black',
    marker_line_width=1,
    selector=dict(type='histogram')
)

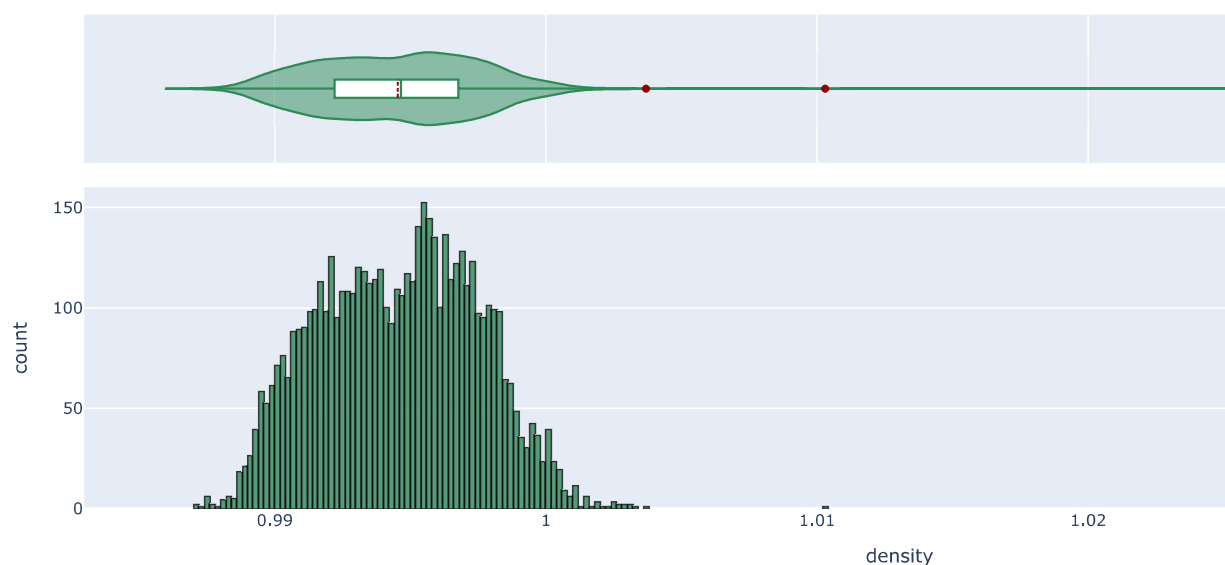
fig.update_layout(
    yaxis2=dict(domain=[0.70, 1.0]),
    yaxis=dict(domain=[0.0, 0.65]),
    bargap=0.1,
    autosize=True,
    height=550
)

fig.show(config={'responsive': True})
```

Grupo 1 - Variáveis Contínuas de Tendência Central: pH



Grupo 1 - Variáveis Contínuas de Tendência Central: density



Percepções

- Tanto o `pH` quanto a `density` concentram a imensa maioria das suas observações em um formato simétrico de "sino". A média (linha tracejada vermelha) e a mediana (linha contínua) estão extremamente próximas, confirmando uma forte tendência central.
- No histograma da `density`, percebe-se um topo ligeiramente "achatado/bimodal" reflexo da mistura das duas categorias no dataset (vinhos vermelhos e brancos possuem densidades levemente distintas).
- A densidade do vinho é diretamente afetada pelo teor de açúcar e álcool. Vinhos doces/sobremesa retêm alta densidade, o que explica os pontos extremos sem necessariamente indicar erro de digitação.
- **Para o KNN:** A presença dos outliers extremos em `density` torna o uso de `StandardScaler` ou `RobustScaler` estritamente obrigatório. Sem o escalonamento, esses valores de densidade distorcerão os cálculos de distância euclidiana.
- **Para a Random Forest:** O algoritmo lidará com ambas as variáveis nativamente sem necessidade de remoção ou transformação prévia.

3.2.2 residual sugar, chlorides e free_sulfur_dioxid

```
# Assimetria Severa
grupo_2 = ['residual_sugar', 'chlorides', 'free_sulfur_dioxide']
```

```
for feature in grupo_2:
    fig = px.histogram(
        wine_uniq_df,
        x=feature,
        marginal='violin',
        title=f'Grupo 2 - Assimetria Severa: {feature}',
        color_discrete_sequence=['#9932CC'],
        opacity=0.8
    )

    # Estilização Violin
    fig.update_traces(
        box_visible=True,
        box_fillcolor='white',
        meanline_visible=True,
        meanline_color='darkred',
        line_color='#9932CC',
        line_width=1.5,
        marker_symbol='circle',
        marker_color='#8B0000',
        selector=dict(type='violin')
    )

    # Estilização Histogram
    fig.update_traces(
        marker_line_color='black',
        marker_line_width=1,
        selector=dict(type='histogram')
    )

    fig.update_layout(
        yaxis2=dict(domain=[0.70, 1.0]),
        yaxis=dict(domain=[0.0, 0.65]),
        bargap=0.1,
        autosize=True,
        height=550
    )

fig.show(config={'responsive': True})
```


Grupo 2 - Assimetria Severa: residual_sugar

Percepções

- Todas as variáveis deste grupo apresentam concentração massiva de valores no limite inferior, com uma cauda longa se estendendo no sentido positivo.
- Em todos os violinos, a linha tracejada vermelha (Média) está visivelmente deslocada à direita em relação à mediana (centro da caixa branca). Isso demonstra como os valores extremos inflam a média aritmética.
- Picos secundários nos histogramas marcam a diferença fundamental entre tipos de vinho:
 - Açúcar:** Vinhos secos (pico primário em torno de 1-3 g/L) vs. vinhos doces/suaves (platô em torno de 15-20 g/L).
 - Cloretos:** Vinhos brancos tendem a ter níveis menores e mais homogêneos, enquanto os vermelhos exibem maior dispersão salina.
- Os pontos isolados no extremo da escala podem representar produtos de nicho.
- KNN:** Por apresentarem distribuições altamente assimétricas e com grandes amplitudes de valores, o escalonamento simples (StandardScaler) pode ter sua média/desvio-padrão distorcidos pelos extremos.
 - O uso de **RobustScaler** ou a aplicação prévia de uma transformação logarítmica ajudará a normalizar estas variáveis para o KNN.
- Random Forests:** Árvores de decisão não apresentam grandes problemas à assimetria severa e outliers.

3.2.3 fixed_acidity, volatile_acidity e sulphates

```
# Assimetria Moderada
grupo_3 = ['fixed_acidity', 'volatile_acidity', 'sulphates']

for feature in grupo_3:
    fig = px.histogram(
        wine_uniq_df,
        x=feature,
        marginal='violin',
        title=f'Grupo 3 - Assimetria Moderada: {feature}',
        color_discrete_sequence=['#E06D53'],
        opacity=0.8
    )

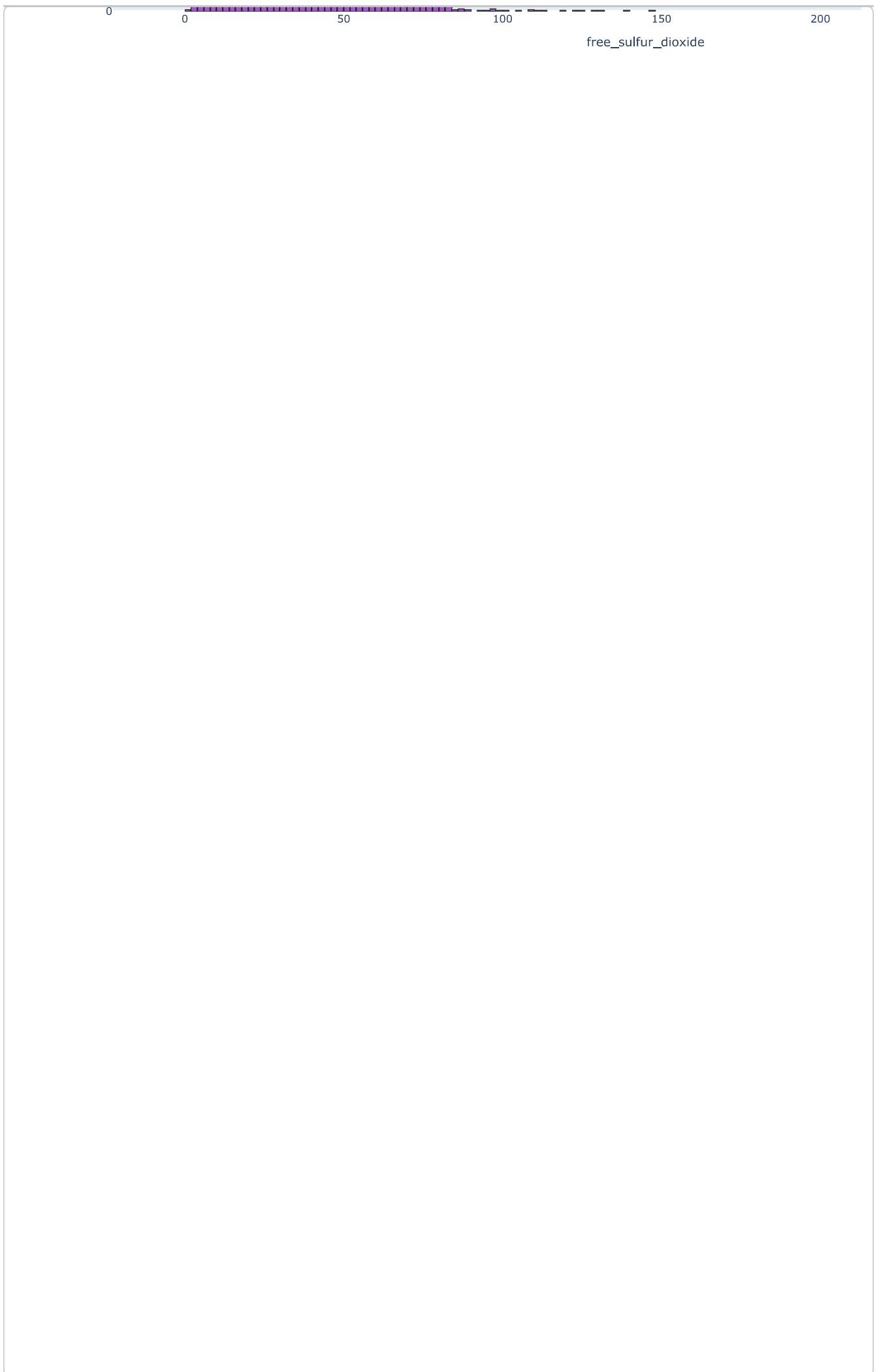
    # Estilização Violin
    fig.update_traces(
        box_visible=True,
        box_fillcolor='white',
        meanline_visible=True,
        meanline_color='darkred',
        line_color='#E06D53',
        line_width=1.5,
        marker_symbol='circle',
        marker_color='#8B0000',
        selector=dict(type='violin')
    )

    # Estilização Histogram
    fig.update_traces(
        marker_line_color='black',
        marker_line_width=1,
        selector=dict(type='histogram')
    )

    fig.update_layout(
        yaxis2=dict(domain=[0.70, 1.0]),
        yaxis=dict(domain=[0.0, 0.65]),
        bargap=0.1,
        autosize=True,
        height=550
    )

fig.show(config={'responsive': True})
```





Grupo 3 - Assimetria Moderada: fixed_acidity

Percepções

- As três variáveis apresentam o pico principal bem definido na metade esquerda da escala, caindo de forma suave até formar uma cauda longa contínua à direita.
- Em todas as três variáveis, a linha tracejada vermelha (Média) situa-se ligeiramente à direita da mediana e com divergência menor do que observada no Grupo 2, confirmando uma assimetria moderada.
- O histograma da acidez volátil mostra uma pequena elevação secundária em torno de 0.6 – 0.7, que indica novamente a mistura do dataset (vinhos tintos tendem a apresentar maior acidez volátil do que os vinhos brancos).
- Amostras com alta acidez volátil ([acima de 0.6 a 0.8 g/L para brancos e de 0.8 a 1.2 g/L para tintos](#)) são casos raros e indicam maior probabilidade de defeito sensorial (gosto a Vinagre/ácido acético).
- **KNN:** Como os dados não têm distorções extremas como a densidade do Grupo 1 ou o açúcar do Grupo 2, o **StandardScaler** tradicional funcionará perfeitamente nestas três variáveis para igualar as amplitudes.
- **Para a Random Forest:** A ligeira assimetria não interfere no desempenho do algoritmo baseado em árvores.

3.2.4 citric_acid, total_sulfur_dioxide, alcohol

fixed_acidity

```
# Padrões Complexos
grupo_4 = ['citric_acid', 'total_sulfur_dioxide', 'alcohol']

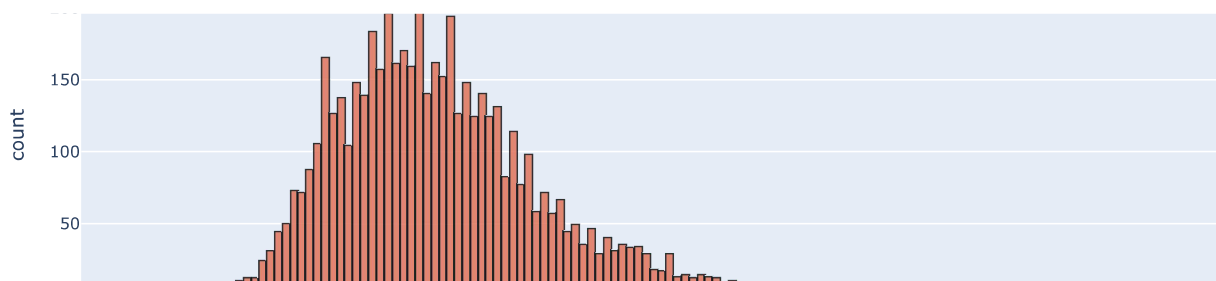
for feature in grupo_4:
    fig = px.histogram(
        wine_uniq_df,
        x=feature,
        marginal='violin',
        title=f'Grupo 4 - Comportamento Complexo / Multimodal: {feature}',
        color_discrete_sequence=['#20B2AA'],
        opacity=0.8
    )

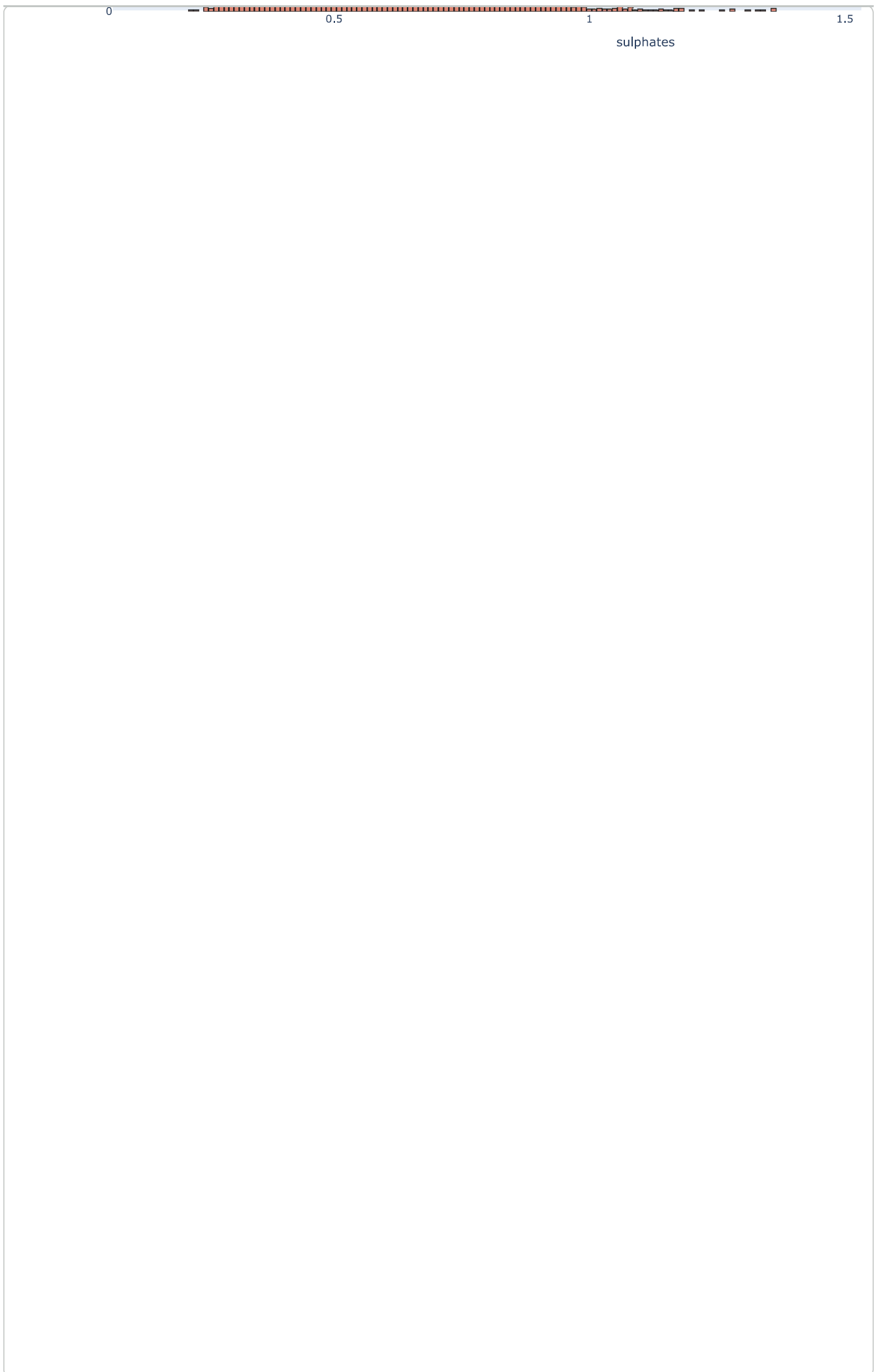
    # Estilização Violin
    fig.update_traces(
        box_visible=True,
        box_fillcolor='white',
        meanline_visible=True,
        meanline_color='darkred',
        line_color='#20B2AA',
        line_width=1.5,
        marker_symbol='circle',
        marker_color='#8B0000',
        selector=dict(type='violin')
    )

    # Estilização Histogram
    fig.update_traces(
        marker_line_color='black',
        marker_line_width=1,
        selector=dict(type='histogram')
    )

    fig.update_layout(
        yaxis2=dict(domain=[0.70, 1.0]),
        yaxis=dict(domain=[0.0, 0.65]),
        bargap=0.1,
        autosize=True,
        height=550
    )

fig.show(config={'responsive': True})
```





Grupo 4 - Comportamento Complexo / Multimodal: citric_acid

Percepções

- `total_sulfur_dioxide` apresenta dois picos que indicam os dois tipos de vinho: O primeiro agrupamento ($< 50 \text{ mg/dm}^3$) corresponde aos vinhos vermelhos, enquanto o segundo agrupamento ($> 100 \text{ mg/dm}^3$) representa os vinhos brancos. [How Much Sulfur is in Wine?](#)
 - Assim, os outliers ($+200$) indicam a presença de vinhos doces, equilibrando o açúcar residual exercendo função antioxidante, antibacteriana e conservatória ([Sulfitos e vinho](#))
- `alcohol` é a variável mais distribuída do grupo, com apenas um ponto discrepante extremo próximo a 15.
- **KNN**: A variável `total_sulfur_dioxide` possui valores que passam de 200, enquanto `citric_acid` opera na escala de 0 a 1. A padronização (`StandardScaler`) é indispensável para que o SO_2 não domine completamente o cálculo de distância do KNN.
 - A multimodalidade destas variáveis sugere que a inclusão da variável `color` (ou o split entre vinhos brancos e tintos) ajudará o KNN a separar melhor esses agrupamentos no espaço de atributos.
- **Para a Random Forest**: Árvores de decisão capturam distribuições multimodais e picos discretos com extrema facilidade através de cortes sucessivos (`splits`), tornando a Random Forest um modelo adequado para este grupo de variáveis.

citric_acid

3.2.5 quality e color

```
# Targets
grupo_5 = ['quality', 'color']

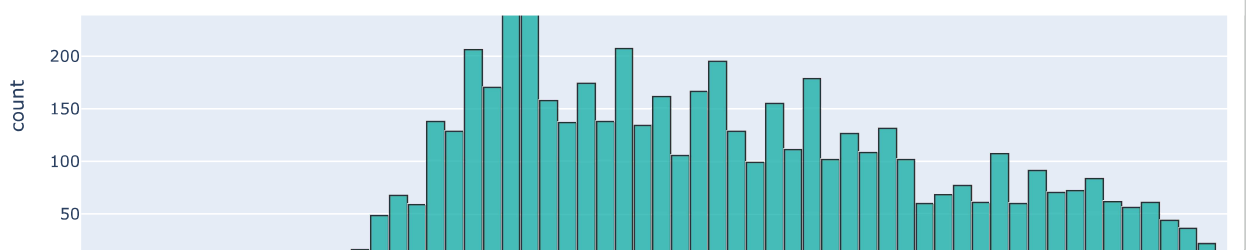
for feature in grupo_5:
    fig = px.histogram(
        wine_uniq_df,
        x=feature,
        marginal='violin',
        title=f'Grupo 5 - Alvos: {feature}',
        color_discrete_sequence=['#775777'],
        opacity=0.8
    )

    # Estilização Violin
    fig.update_traces(
        box_visible=True,
        box_fillcolor='white',
        meanline_visible=True,
        meanline_color='darkred',
        line_color='#775777',
        line_width=1.5,
        marker_symbol='circle',
        marker_color='#8B0000',
        selector=dict(type='violin')
    )

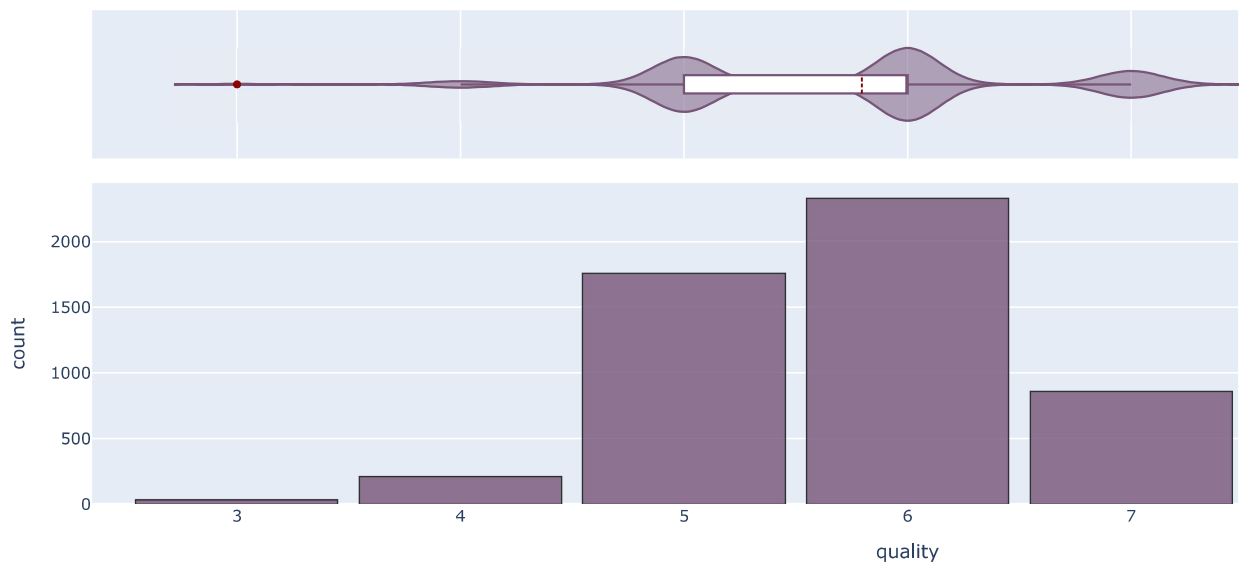
    # Estilização Histogram
    fig.update_traces(
        marker_line_color='black',
        marker_line_width=1,
        selector=dict(type='histogram')
    )

    fig.update_layout(
        yaxis2=dict(domain=[0.70, 1.0]),
        yaxis=dict(domain=[0.0, 0.65]),
        bargap=0.1,
        autosize=True,
        height=550
    )

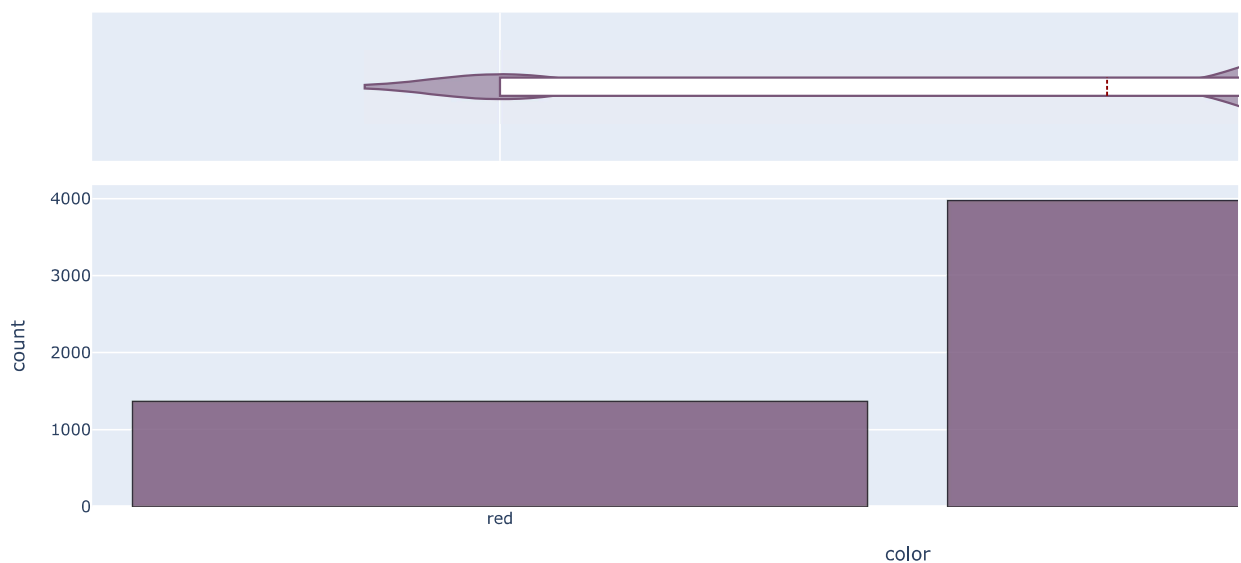
fig.show(config={'responsive': True})
```



Grupo 5 - Alvos: quality



Grupo 5 - Alvos: color



Percepções

- O dataset é maioritariamente composto por vinhos brancos (`white` ~ 75%) em comparação aos vinhos vermelhos (`red` ~ 25%).
- A imensa maioria das avaliações situa-se nas notas **5 e 6** (que juntas representam mais de 70% do dataset).
- **Escassez de Classes Extremas:** As notas de qualidade extrema (**3, 4, 8 e 9**) contêm pouquíssimas amostras.
- Na divisão de dados (`train_test_split`), será importante estratificar (`stratify=y`) para garantir que as classes raras fiquem proporcionalmente representadas tanto no treino como no teste.
- **KNN e Random Forest:** Avaliar os modelos através de **Macro F1-Score, Precision e Recall**, e/ou através da matriz de confusão normalizada.

3.3 Análise Bivariada e Multivariada

Avaliação da interação entre as variáveis.

```
# Matriz de correlação apenas com variáveis numéricas
df_eda = wine_uniq_df.copy()
```

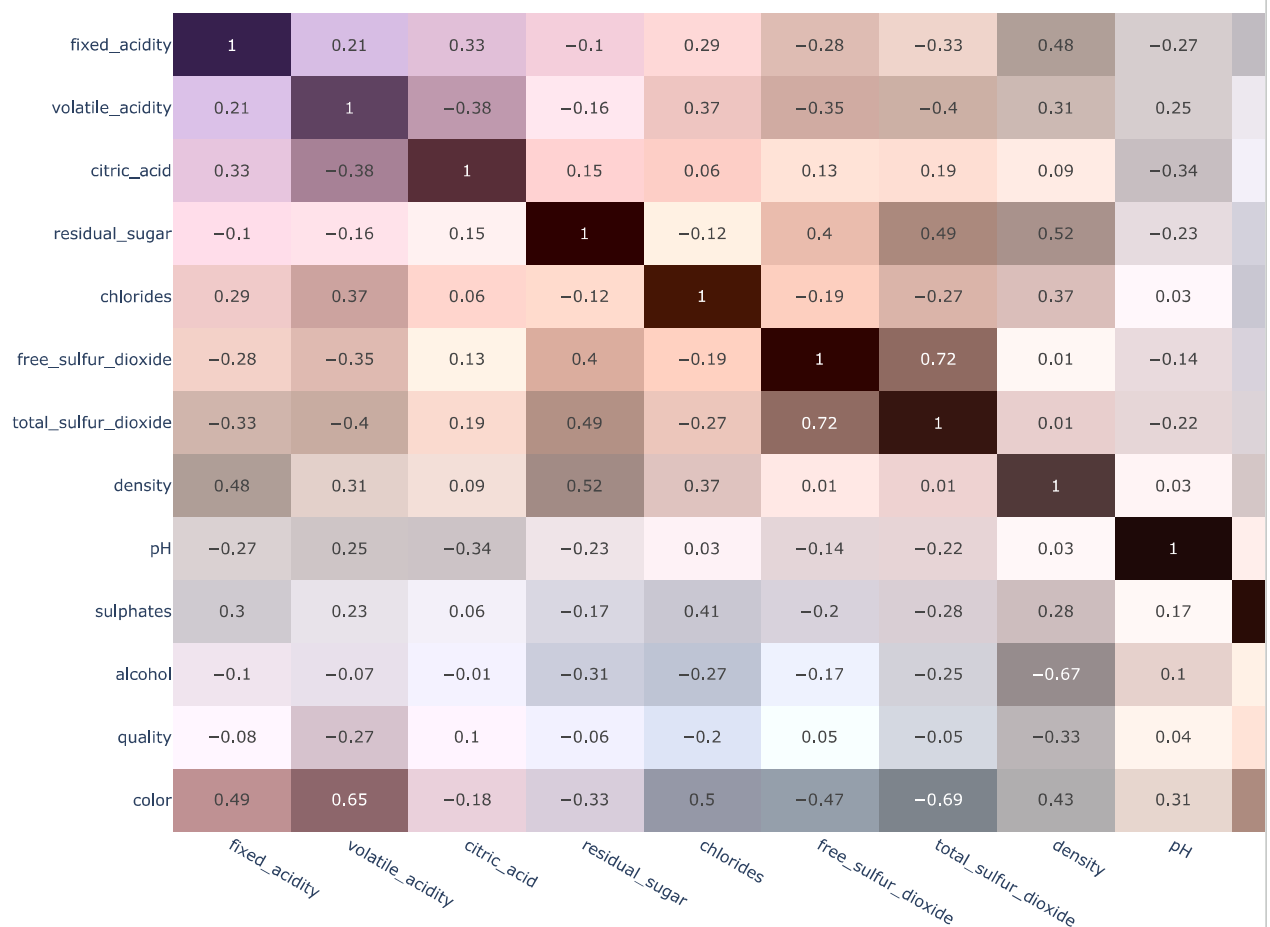
```
# Encoding Binário (0 = white, 1 = red)
df_eda['color'] = df_eda['color'].map({'white': 0, 'red': 1})

# Matriz de correlação
matriz_corr = df_eda.corr().round(2)

# Heatmap
fig_corr = px.imshow(
    matriz_corr,
    text_auto=True,
    aspect="auto",
    color_continuous_scale='RdBu_r',
    zmin=-1, zmax=1,
    title='Matriz de Correlação Linear (Incluindo Cor e Qualidade)'
)

fig_corr.update_layout(height=800, autosize=True)
fig_corr.show(config={'responsive': True})
```

Matriz de Correlação Linear (Incluindo Cor e Qualidade)

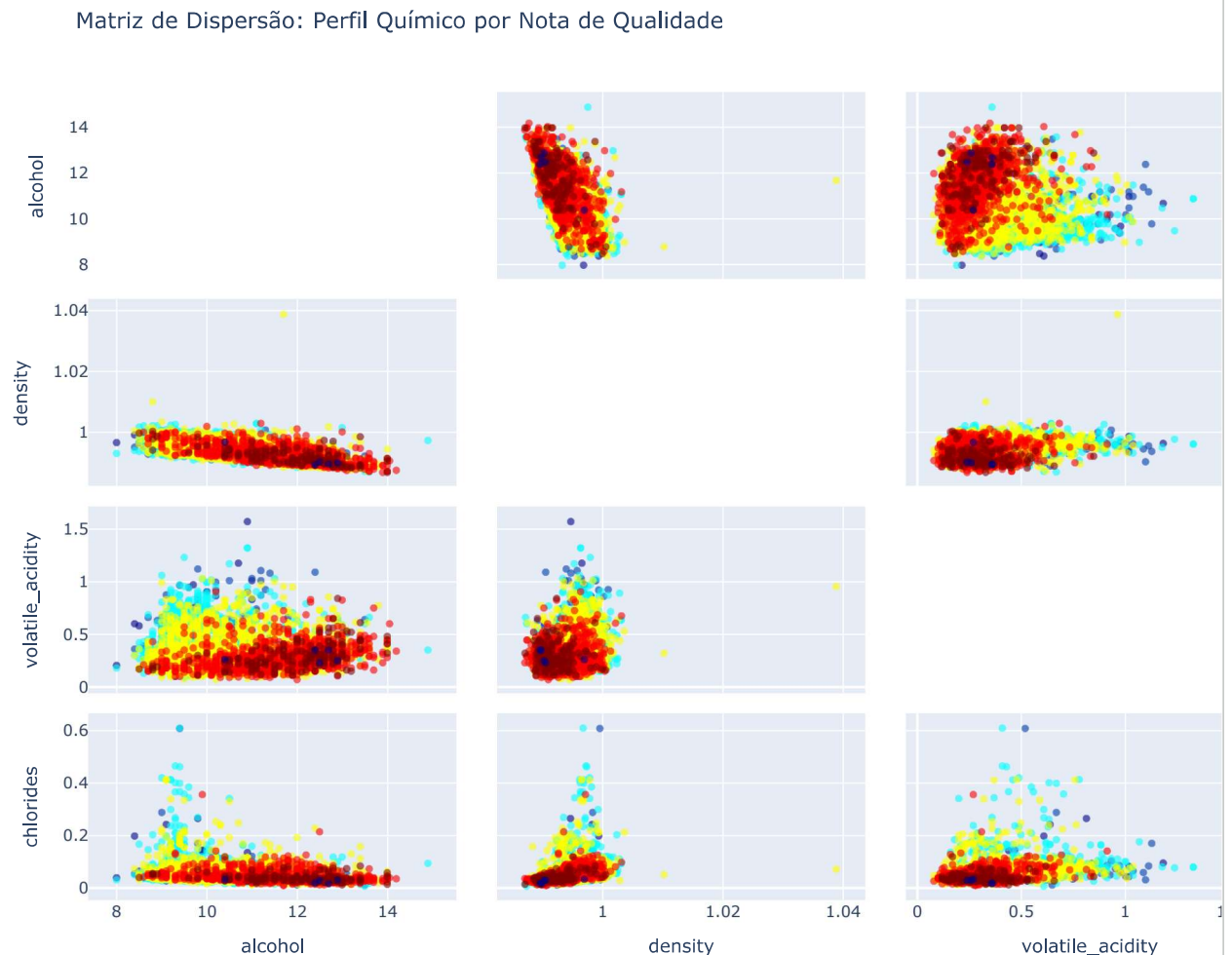


- `free_sulfur_dioxide` e `total_sulfur_dioxide` possuem forte correlação linear, o que já se era esperado por se tratar de componente e soma de dióxido sulfurico.
- Ambos os *targets* `quality` e `color` possuem fraca correlação, indicando que há pouco impacto entre a cor e a qualidade avaliada, o que permite análises mais independentes.
- Para `quality`, aparentemente os principais *features* a se observar seriam `alcohol`, `density` e `volatile_acidity` (níveis elevados levam à semelhança do vinagre).
 - `density` se relaciona inversamente com o próprio percentual de `alcohol` e diretamente com `residual_sugar` (nível de doçura).
- Para `color`, aparentemente os principais *features* a se observar seriam `total_sulfur_dioxide` (níveis elevados levam a um gosto indesejado), `volatile_acidity` (níveis elevados levam à semelhança do vinagre) e `chlorides` (sais dissolvidos).

```
# Key_features X Quality
key_features_quality = ['alcohol', 'density', 'volatile_acidity', 'chlorides']
```

```
fig_matrix_quality = px.scatter_matrix(
    df_eda,
    dimensions=key_features_quality,
    color=df_eda['quality'].astype(str),
    title='Matriz de Dispersão: Perfil Químico por Nota de Qualidade',
    opacity=0.6,
    category_orders={"color": ["3", "4", "5", "6", "7", "8", "9"]},
    color_discrete_sequence=px.colors.sequential.Jet
)

# Oculta a diagonal principal
fig_matrix_quality.update_traces(diagonal_visible=False)
fig_matrix_quality.update_layout(height=800, autosize=True)
fig_matrix_quality.show(config={'responsive': True})
```



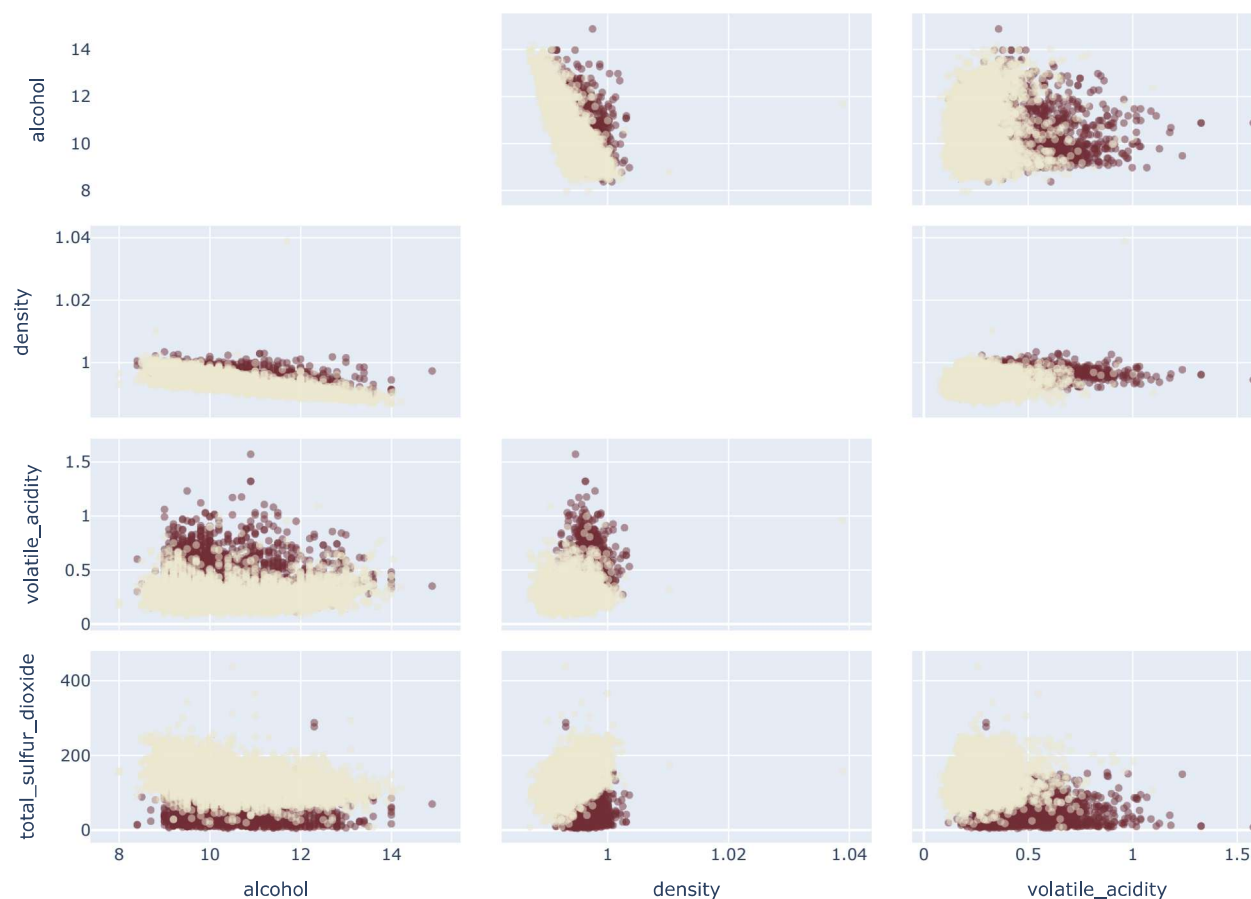
Há um espalhamento bagunçado entre as principais features e seus valores de target, indicando que a previsão da nota do vinho não obedece a regras lineares simples.

```
# Key_features X Color
key_features_color = ['alcohol', 'density', 'volatile_acidity', 'total_sulfur_dioxide']

fig_matrix = px.scatter_matrix(
    df_eda,
    dimensions=key_features_color,
    color=df_eda['color'].map({0: 'white', 1: 'red'}),
    title='Matriz de Dispersão: Perfil Químico por Cor',
    opacity=0.5,
    color_discrete_map=WINE_COLORS
)

# Oculta a diagonal principal para o gráfico ficar mais limpo
fig_matrix.update_traces(diagonal_visible=False)
fig_matrix.update_layout(height=800, autosize=True)
fig_matrix.show(config={'responsive': True})
```

Matriz de Dispersão: Perfil Químico por Cor



As principais *features* quando relacionadas em pares apresentam regiões bem-definidas, mas ainda sim com sobreposição de dados. Isso sugere que a predição do tipo de vinho é matematicamente viável, mas exige uma capacidade de avaliar múltiplas variáveis simultaneamente e segmentar as amostras que caem nas zonas de intersecção química.

✓ Percepções Pré-ML

Relação Físico-Química e Multicolinearidade: A `density` atua como uma variável dependente do binômio Álcool vs. Açúcar. Vinhos secos e com elevado teor alcoólico apresentam densidades muito baixas, ao passo que vinhos doces (*outliers* de `residual_sugar` > 15 g/L) elevam significativamente a densidade. Como `density`, `residual_sugar` e `alcohol` apresentam forte multicolinearidade e outliers extremos, o `StandardScaler` ou `RobustScaler` é obrigatório para o **KNN** não ser distorcido por estas amplitudes.

Assinatura Físico-Química por Tipo de Vinho: A bimodalidade acentuada em `total_sulfur_dioxide` e a assimetria em `free_sulfur_dioxide` são explicadas pelo desequilíbrio de classes da variável `color` (75% brancos). Vinhos brancos exigem doses substancialmente maiores de sulfitos para evitar a oxidação e preservar a cor clara. A introdução da variável `color` no treino (ou a padronização das feições de enxofre) permitirá aos algoritmos (especialmente o **KNN**) separar com clareza estas duas subpopulações.

Impacto Organoléptico no Alvo (quality): A acidez volátil elevada (> 0.8 g/L) é um indicador crítico de degradação acética (sabor a vinagre). A EDA revela que as poucas amostras com acidez volátil alta correlacionam-se com as notas mais baixas da variável alvo `quality` (notas 3 e 4). Como as classes extremas de qualidade são raras, a divisão dos dados com `stratify=y` no `train_test_split` é indispensável.

✓ 4. Pré-processamento e Engenharia de Features (Adaptação)

```
df_original = wine_df.copy()
df_limpo = wine_uniq_df.copy()
df_original['color'] = wine_df['color'].map({'red': 0, 'white': 1})
df_limpo['color'] = wine_uniq_df['color'].map({'red': 0, 'white': 1})
```

▼ Quality x Color

```
# Dicionário organizando as duas bases de dados
datasets = {
    'Com Duplicadas': wine_df,
    'Sem Duplicadas': wine_uniq_df
}

# 1. Agrupamento Estatístico
lista_estaticas = [df.groupby('color')['quality'].describe() for df in datasets.values()]
estatisticas_comparacao = pd.concat(lista_estaticas, keys=datasets.keys())

print("### Resumo Estatístico da Qualidade por Cor\n")
display(
    estatisticas_comparacao
    .round(2)
    .style
    .format("{:.2f}")
    .background_gradient(cmap="Blues", subset=pd.IndexSlice[:, ['mean', 'std']])
    .set_properties(**{'text-align': 'center'})
    .set_caption("Comparação: Com Duplicadas x Sem Duplicadas")
)

print("\n" + "-" * 80 + "\n")

# 2. Tabela de Contingência Proporcional
lista_distribuicao = [
    (pd.crosstab(index=df['color'], columns=df['quality'], normalize='index') * 100).round(2)
    for df in datasets.values()
]
distribuicao_comparacao = pd.concat(lista_distribuicao, keys=datasets.keys()).fillna(0)

print("### Distribuição Percentual das Notas de Qualidade por Cor\n")
display(
    distribuicao_comparacao
    .style
    .format("{:.1f}%")
    .background_gradient(cmap="YlGnBu", axis=1)
    .set_properties(**{'text-align': 'center'})
    .set_caption("Valores em % (por cor)")
)
```

Resumo Estatístico da Qualidade por Cor

		Comparação: Com Duplicadas x Sem Duplicadas							
		count	mean	std	min	25%	50%	75%	max
color									
Com Duplicadas	red	1599.00	5.64	0.81	3.00	5.00	6.00	6.00	8.00
	white	4898.00	5.88	0.89	3.00	5.00	6.00	6.00	9.00
Sem Duplicadas	red	1359.00	5.62	0.82	3.00	5.00	6.00	6.00	8.00
	white	3961.00	5.85	0.89	3.00	5.00	6.00	6.00	9.00

Distribuição Percentual das Notas de Qualidade por Cor

		Valores em % (por cor)							
		quality	3	4	5	6	7	8	9
color									
Com Duplicadas	red	0.6%	3.3%	42.6%	39.9%	12.4%	1.1%	0.0%	
	white	0.4%	3.3%	29.8%	44.9%	18.0%	3.6%	0.1%	
Sem Duplicadas	red	0.7%	3.9%	42.5%	39.4%	12.3%	1.2%	0.0%	
	white	0.5%	3.9%	29.7%	45.1%	17.4%	3.3%	0.1%	

- Apparently the dataframes with and without duplicates should not constitute a large change in the model's classification. Still, it is worth comparing both cases.
- `quality` will be removed for the color classification as it only counts attributes.
- Another important point to note is that there are no identifiers, be it the wine, the evaluator, or even the test. This makes `quality` much more subjective, beyond not being reliable that the duplicated lines are just coincidences.

Análise de Componentes Principais (PCA)

O PCA é uma técnica estatística não supervisionada que reduz o número de dimensões em grandes conjuntos de dados, sendo eficaz para visualizar e explorar conjuntos de dados de alta dimensão ou dados com muitos atributos, pois pode identificar facilmente tendências, padrões ou valores discrepantes.

```
stroke_map = {
    'red': '#B9979B', # Borda clara para o tinto
    'white': '#777468' # Borda escura para o branco
}

# 1. Isolando as features químicas e a variável alvo
X = df_limpo.drop(columns=['color', 'quality'], errors='ignore')
y = df_limpo['color']

# 2. Padronizando os dados
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# --- PCA 2D ---
pca_2d = PCA(n_components=2)
components_2d = pca_2d.fit_transform(X_scaled)

df_pca_2d = pd.DataFrame(data=components_2d, columns=['PC1', 'PC2'])
df_pca_2d['color'] = y.values
df_pca_2d['color'] = df_pca_2d['color'].map({0: 'white', 1: 'red'})

display(df_pca_2d.head())

# Gráfico 2D interativo com Plotly
fig_2d = px.scatter(
    df_pca_2d, x='PC1', y='PC2', color='color',
    title=f'PCA 2D - Variância Explicada Total: {pca_2d.explained_variance_ratio_.sum()*100:.2f}%',
    color_discrete_map=WINE_COLORS,
    opacity=0.8
)

for trace in fig_2d.data:
    classe = trace.name # 'red' ou 'white'
    if classe in stroke_map:
        trace.marker.line.width = 1
        trace.marker.line.color = stroke_map[classe]

fig_2d.show()

# --- PCA 3D ---
pca_3d = PCA(n_components=3)
components_3d = pca_3d.fit_transform(X_scaled)

df_pca_3d = pd.DataFrame(data=components_3d, columns=['PC1', 'PC2', 'PC3'])
df_pca_3d['color'] = y.values
df_pca_3d['color'] = df_pca_3d['color'].map({0: 'white', 1: 'red'})

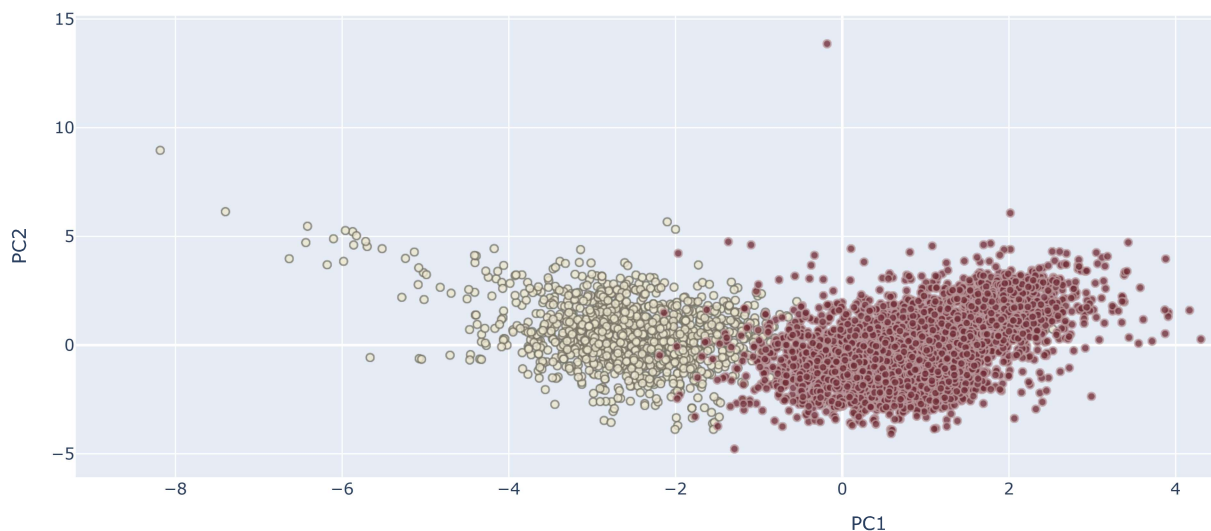
# Gráfico 3D interativo com Plotly
fig_3d = px.scatter_3d(
    df_pca_3d, x='PC1', y='PC2', z='PC3', color='color',
    title=f'PCA 3D - Variância Explicada Total: {pca_3d.explained_variance_ratio_.sum()*100:.2f}%',
    color_discrete_map=WINE_COLORS,
    opacity=0.8
)

for trace in fig_3d.data:
    classe = trace.name # 'red' ou 'white'
    if classe in stroke_map:
        trace.marker.line.width = 1
        trace.marker.line.color = stroke_map[classe]

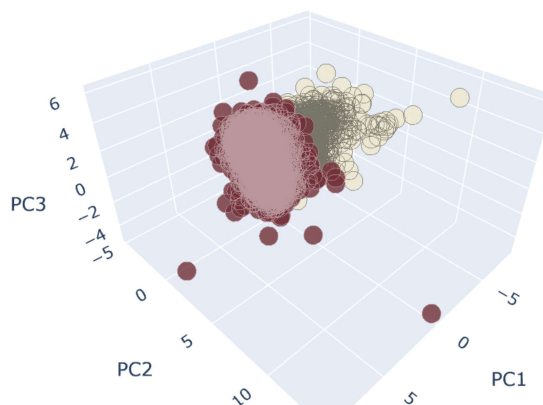
fig_3d.show()
```

	PC1	PC2	color
0	-3.165753	-0.010419	white
1	-3.054962	0.724687	white
2	-3.072476	0.485413	white
3	-1.756467	1.959557	white
4	-2.972872	-0.009583	white

PCA 2D - Variância Explicada Total: 49.67%



PCA 3D - Variância Explicada Total: 64.10%



- Observa-se uma que, tanto no gráfico 2D quanto no 3D, os vinhos tintos (red) e brancos (white) formam dois agrupamentos (clusters) bem definidos.
- O PC1 (eixo horizontal no gráfico 2D) é claramente a dimensão responsável por separar a cor: Vinhos brancos concentram-se em valores negativos, enquanto os vinhos tintos concentram-se em valores positivos.
- Para um dataset multidimensional e complexo, conseguir separar quase perfeitamente duas classes utilizando de aproximadamente 50% a 64% da variância total (quão distantes os valores de um conjunto de dados estão em relação à média aritmética) demonstra que as variáveis de maior variância do dataset estão diretamente correlacionadas ao tipo de vinho.

Preparação para Classificadores de Cor

```
df_original = wine_df.copy()
df_limpo = wine_uniq_df.copy()
df_original['color'] = wine_df['color'].map({'red': 0, 'white': 1})
df_limpo['color'] = wine_uniq_df['color'].map({'red': 0, 'white': 1})
```

```
def gerar_splits_estratificados(
    df: pd.DataFrame,
    coluna_alvo: str,
    n_splits: int = 11,
    random_state: int = 33
) -> list[tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series]]:
    """
    Gera partições de treino e teste estratificadas para validação cruzada,
    garantindo a proporção das classes minoritárias em todos os 'folds'.

    Args:
        df (pd.DataFrame): O DataFrame completo contendo as features e o alvo.
            recebe os dfs com e sem duplicada
        coluna_alvo (str): O nome da coluna que será utilizada como alvo (target).
            espera-se quality ou color
        n_splits (int, optional): O número de partições (folds) para a divisão.
            default=5.
        random_state (int, optional): Semente aleatória para reprodutibilidade.
            default = 33.

    Returns:
        list[tuple[pd.DataFrame, pd.DataFrame, pd.Series, pd.Series]]:
            Uma lista contendo `n_splits` tuplas. Cada tupla representa um 'fold'
            e contém os seguintes elementos, nesta exata ordem:
            - X_train (pd.DataFrame): Features de treinamento.
            - X_test (pd.DataFrame): Features de teste.
            - y_train (pd.Series): Alvo de treinamento.
            - y_test (pd.Series): Alvo de teste.
    """

    # Separação das features e do alvo
    X = df.drop(columns=[coluna_alvo])
    y = df[coluna_alvo]

    # Instanciando o separador estratificado
    skf = StratifiedKFold(n_splits=n_splits, shuffle=True, random_state=random_state)
    splits = []

    # Gerando os índices e construindo as partições
    for train_index, test_index in skf.split(X, y):
        X_train = X.iloc[train_index]
        X_test = X.iloc[test_index]
        y_train = y.iloc[train_index]
        y_test = y.iloc[test_index]

        splits.append((X_train, X_test, y_train, y_test))

    return splits
```

```
def calcular_hash_pandas(obj: pd.DataFrame | pd.Series) -> str:
    """
    Gera um hash MD5 único para um DataFrame ou Series do Pandas,
    garantindo a verificação de integridade dos dados.
    """
    # Hash de cada linha do objeto
    hashes_linhas = pd.util.hash_pandas_object(obj, index=True)
    # Combina todos os hashes em uma única string hexadecimal (MD5)
    return hashlib.md5(hashes_linhas.values).hexdigest()
```

▼ Treino e Teste

```
splits_com_duplicadas = gerar_splits_estratificados(df_original, coluna_alvo='color')
splits_sem_duplicadas = gerar_splits_estratificados(df_limpo, coluna_alvo='color')

# Dicionário prático para o loop de treino
cenarios_cor = {
    'Com Duplicadas': splits_com_duplicadas,
    'Sem Duplicadas': splits_sem_duplicadas
}
```

```
for cenario, splits in cenarios_cor.items():
    print(f" === CENÁRIO: {cenario.upper()} === ")
```